

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт з лабораторної роботи №2
з дисципліни: «Методи оптимізації та теорія ігор»
з теми: «Програмна реалізація методів підтримки індивідуального прийняття
рішень»

Виконав:

ст. гр. ПЗПІ-23-8

Потурайко Н.І.

Варіант: №7

Перевірів:

асистент каф. ПІ

Демчук В. Г.

2 ПРОГРАМНА РЕАЛІЗАЦІЯ МЕТОДІВ ПІДТРИМКИ ІНДИВІДУАЛЬНОГО ПРИЙНЯТТЯ РІШЕНЬ

1.1 Мета роботи

Закріпити теоретичні знання з методів прийняття рішень у слабкоструктурованих предметних областях. Придбати практичні навички програмної реалізації методів підтримки особи, що приймає рішення, за допомогою апарату бінарних відношень. Навчитися будувати матриці суміжності на основі попарних порівнянь та знаходити оптимальні рішення шляхом пошуку максимального елемента.

1.2 Опис обраного методу прийняття рішень

Для реалізації індивідуального прийняття рішень було обрано метод на основі бінарних відношень, а саме пошук рішення як максимального елемента матриці суміжності. Цей підхід є одним з найбільш фундаментальних у теорії прийняття рішень, оскільки він дозволяє формалізувати суб'єктивні переваги експерта через послідовні елементарні акти вибору.

Суть методу полягає у проведенні повної серії попарних порівнянь усіх наявних альтернатив. Для множини з шести ігрових рушіїв загальна кількість таких порівнянь складає п'ятнадцять. Під час кожного порівняння експерт висловлює свою перевагу одній з альтернатив або констатує їх рівноцінність.

Математично результат порівнянь відображається у матриці суміжності. Елемент матриці на перетині i -го рядка та j -го стовпця набуває значення один, якщо альтернатива i є кращою за альтернативу j за сукупністю критеріїв. В усіх інших випадках елемент дорівнює нулю. Максимальним елементом вважається така альтернатива, для якої у відповідному рядку матриці суміжності всі значення дорівнюють одиниці, за винятком головної діагоналі. Це означає, що дана альтернатива домінує над усіма іншими варіантами у множині.

1.3 Алгоритм програмної реалізації

Програмна реалізація методу була інтегрована у розроблений раніше веб-застосунок. Алгоритм роботи модуля прийняття рішень розділений на декілька логічних етапів.

На першому етапі система ініціалізує сесію, дозволяючи обрати експерта з бази даних. Це необхідно для персоналізації результатів та подальшого колективного аналізу.

На другому етапі виконується генерація всіх можливих унікальних пар альтернатив. Програма послідовно виводить на екран пари ігрових рушіїв разом з їхніми векторними оцінками, що були внесені під час першої лабораторної роботи. Експерт має можливість порівняти характеристики та натиснути кнопку вибору. Кожна дія фіксується у таблиці історії порівнянь.

На третьому етапі на основі зібраних відповідей автоматично будується матриця суміжності розміром шість на шість. Програма виконує перевірку кожного рядка на відповідність умові максимального елемента. Якщо такий елемент знайдено, він отримує статус переможця та автоматично посідає перше місце у підсумковому рейтингу.

На заключному етапі система вимагає від користувача встановити строгий порядок для решти альтернатив. Це забезпечує цілісність даних для майбутніх розрахунків. Усі отримані ранги зберігаються у реляційній базі даних.

1.4 Оновлена модель бази даних

Для підтримки функціоналу другої лабораторної роботи до структури бази даних було додано нову таблицю `Comparison_History`. Вона призначена для аудиту дій експерта та збереження проміжних результатів вибору.

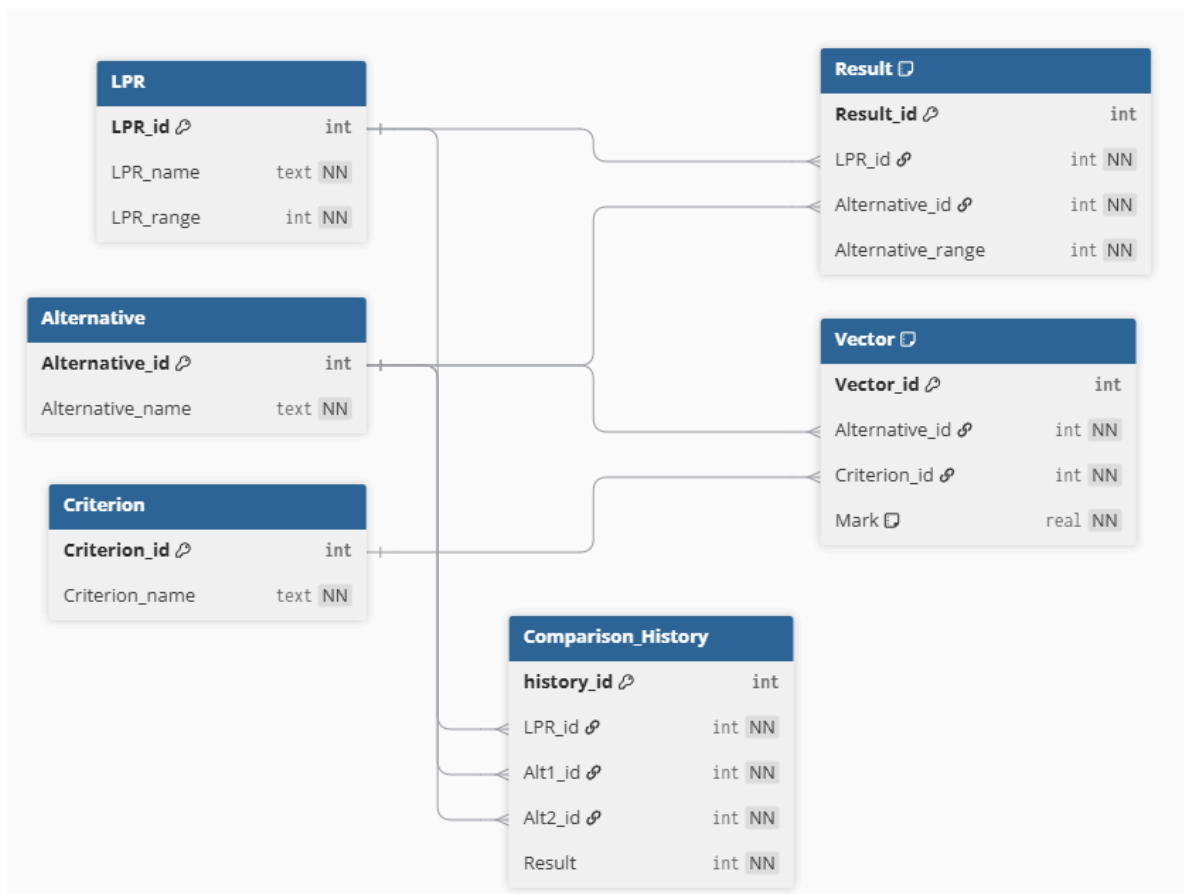


Рисунок 1.1 – Оновлена логічна структура бази даних системи

Таблиця зберігає ідентифікатор експерта, посилання на обидві альтернативи та числовий результат порівняння. Використання такої структури дозволяє відтворити хід думок експерта та перевірити його вибір на логічну несуперечливість. Усі зв'язки реалізовані через зовнішні ключі з правилом каскадного видалення для забезпечення посилальної цілісності.

1.5 Екранні форми додатку

Інтерфейс модуля прийняття рішень розроблений з дотриманням принципів сучасного дизайну. На сторінці вибору експерта користувач може розпочати нову сесію.

Прийняття рішення

Етап 1: Вибір експерта

Оберіть експерта (ОПР) для початку попарного порівняння рушіїв. Це дозволить визначити найкращу альтернативу.

Оберіть експерта:

Студент (фактично головний розробник)

Розпочати порівняння

Рисунок 1.2 – Екранна форма вибору експерта для сесії прийняття рішення

Процес попарного порівняння реалізований у вигляді інтерактивних карток. Кожна картка містить детальні дані про рушій, що допомагає експерту приймати обґрунтоване рішення.

Попарне порівняння

Пара 1 з 15

Unity

Легкість освоєння	7
Якість 3D/2D графіки	8
Підтримка спільноти	10
Кросплатформеність	10
Гнучкість розробки	7
Фінансова доступність (Безкоштовність)	6
Екосистема та плагіни	10

Цей краще

VS

Unreal Engine 5

Легкість освоєння	4
Якість 3D/2D графіки	10
Підтримка спільноти	9
Кросплатформеність	9
Гнучкість розробки	9
Фінансова доступність (Безкоштовність)	7
Екосистема та плагіни	9

Цей краще

Рівні по якості

Неспівставні

Рисунок 1.3 – Інтерфейс діалогу попарного порівняння ігрових рушіїв

Після завершення опитування програма візуалізує отриману матрицю суміжності. Це дозволяє наочно побачити структуру переваг та підтвердити правильність знаходження максимального елемента.

Етап 3: Матриця суміжності

Результати всіх попарних порівнянь. Якщо елемент рядка дорівнює 1, це означає, що рушій у рядку переважає або дорівнює рушію у стовпці.

РУШІЙ	UNITY	UNREAL ENGINE 5	GAMEMAKER	DEFOLD	GODOT ENGINE	CRYENGINE
Unity	-	1	1	1	1	1
Unreal Engine 5	0	-	0	1	0	1
GameMaker	0	1	-	0	1	0
Defold	0	0	1	-	1	0
Godot Engine	0	1	0	0	-	1
CryEngine	0	0	1	1	0	-

☒ **Пошук Максимального елемента:**
Знайдено! Рушій **Unity** переважає всі інші альтернативи.

[Перейти до фінального ранжування](#)

Рисунок 1.4 – Візуалізація матриці суміжності та пошук максимального елемента

Фінальна стадія роботи передбачає розстановку всіх альтернатив по місцях. Система контролює, щоб кожен рушій мав унікальний ранг, що є обов'язковою умовою для виконання наступних робіт.

Етап 4: Фінальне ранжування

Максимальний елемент автоматично отримав 1-ше місце. Розставте місця для рушіїв. Ранги повинні бути унікальними від 1 до 6.

РУШІЙ	РАНГ (МІСЦЕ)
Unity Максимальний	1
Unreal Engine 5	2
GameMaker	4
Defold	5
Godot Engine	3
CryEngine	6

[Почати заново](#) [Зберегти результати в БД](#)

Рисунок 1.5 – Форма встановлення строгого порядку та збереження результатів

project > server > database.sqlite

Filter 7 tables... Rows: 45

TABLES

- Alternative
- Comparison_History
- Criterion
- LPR
- Result
- sqlite_sequence
- Vector

histor...	LPR_id	Alt1_id	Alt2_id	Result
15	1	5	6	1
16	3	1	2	1
17	3	1	3	-1
18	3	1	4	1
19	3	1	5	1
20	3	1	6	1
21	3	2	3	-1
22	3	2	4	-1
23	3	2	5	-1
24	3	2	6	1
25	3	3	4	1
26	3	3	5	1
27	3	3	6	1
28	3	4	5	-1
29	3	4	6	-1
30	3	5	6	-1
31	2	1	2	1
32	2	1	3	1
33	2	1	4	1
34	2	1	5	1
35	2	1	6	1
36	2	2	3	-1
37	2	2	4	-1
38	2	2	5	1
39	2	2	6	-1
40	2	3	4	1
41	2	3	5	-1
42	2	3	6	1
43	2	4	5	-1
44	2	4	6	1
45	2	5	6	-1

Рисунок 1.6 – Вміст таблиць Result та Comparison_History після завершення роботи

1.6 Висновки

У ході виконання лабораторної роботи було успішно реалізовано метод підтримки індивідуального прийняття рішень на основі бінарних відношень. Було розширено функціонал веб-застосунку шляхом створення модуля попарних порівнянь та автоматичної генерації матриці суміжності.

Програмний продукт дозволяє ефективно знаходити найкращу альтернативу серед множини ігрових рушіїв через виявлення максимального елемента. Також було забезпечено механізм встановлення строгого порядку всіх варіантів вибору та їх надійне збереження у базі даних SQLite. Отримані результати є необхідним підґрунтям для проведення колективного аналізу у наступній лабораторній роботі.

ДОДАТОК А

Відеодемонстрація роботи додатку

Відеозапис з захистом лабораторної роботи доступний за посиланням: <https://youtu.be/obzpi7QD8pg>

ДОДАТОК Б

Вихідний код проекту

Вихідний код оновленої клієнтської та серверної частин доступний у репозиторії за посиланням: <https://github.com/NurePoturaikoNazar/MOTI-Optimization-Methods-and-Game-Theory>

ДОДАТОК В

Скрипт ініціалізації бази даних

```

PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS LPR (
    LPR_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    LPR_name    TEXT NOT NULL,
    LPR_range  INTEGER NOT NULL CHECK (LPR_range >= 1)
);

CREATE TABLE IF NOT EXISTS Alternative (
    Alternative_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    Alternative_name TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Criterion (
    Criterion_id    INTEGER PRIMARY KEY AUTOINCREMENT,
    Criterion_name  TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Vector (
    Vector_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    Alternative_id  INTEGER NOT NULL,
    Criterion_id    INTEGER NOT NULL,
    Mark           REAL NOT NULL DEFAULT 0,
    FOREIGN KEY (Alternative_id) REFERENCES Alternative(Alternative_id)
ON DELETE CASCADE,
    FOREIGN KEY (Criterion_id) REFERENCES Criterion(Criterion_id) ON
DELETE CASCADE,
    CONSTRAINT UQ_Alternative_Criterion UNIQUE (Alternative_id,
Criterion_id)
);

CREATE TABLE IF NOT EXISTS Result (
    Result_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    LPR_id         INTEGER NOT NULL,
    Alternative_id  INTEGER NOT NULL,
    Alternative_range INTEGER NOT NULL CHECK (Alternative_range >= 1),
    FOREIGN KEY (LPR_id) REFERENCES LPR(LPR_id) ON DELETE CASCADE,
    FOREIGN KEY (Alternative_id) REFERENCES Alternative(Alternative_id)
ON DELETE CASCADE,
    CONSTRAINT UQ_LPR_Alternative UNIQUE (LPR_id, Alternative_id)
);

CREATE TABLE IF NOT EXISTS Comparison_History (
    history_id  INTEGER PRIMARY KEY AUTOINCREMENT,
    LPR_id     INTEGER NOT NULL,
    Alt1_id    INTEGER NOT NULL,
    Alt2_id    INTEGER NOT NULL,
    Result     INTEGER NOT NULL CHECK (Result IN (-1, 0, 1)),
    FOREIGN KEY (LPR_id) REFERENCES LPR(LPR_id) ON DELETE CASCADE,
    FOREIGN KEY (Alt1_id) REFERENCES Alternative(Alternative_id) ON
DELETE CASCADE,
    FOREIGN KEY (Alt2_id) REFERENCES Alternative(Alternative_id) ON
DELETE CASCADE
);

```